Simplifying simple uses of `<random>`

Document number:	P0347R1
Date:	2016-10-16
Audience:	SG6 Numerics, Library Evolution Working Group

Reply-to: R. "Tim" Song <rs2740@gmail.com>, Melissa O'Neill <oneill@cs.hmc.edu>

Abstract

This paper proposes adding to the standard library a class template `random_generator`, which functions as a thin wrapper over an underlying uniform random number generator to simplify seeding and common simple tasks, improving convenience and correctness.

History

R1: Incorporate feedback from Oulu small group discussion.

- Renamed `sample` to `inplace_sample`, reordered parameters to match `std::sample`'s order better, and added additional requirements.
- Added new `sample` to match `std::sample`.
- Removed `sample`-related discussion in the Q&A section.
- Require `DistType` to meet the distribution requirements.
- Explicitly note that the in-place constructor doesn't perform seeding.
- s/URNG/URBG/g to match P0346R0.

Motivation

C++11's `<random>` facility is elegantly designed and easily extensible, but performing even the simplest tasks requires substantial boilerplate that is easy to get wrong, which is a significant problem for beginners, and even seasoned programmers.

Consider the following admittedly silly program written using the proposed facility:

```
#include <random>
#include <iostream>
int main()
{
    std::mt19937_rng rng; // nondeterministically seeded, convenience typedef for std::random_generator<std::mt19937>
    std::cout << "Greetings from Office #" << rng.uniform(1,17) // uniform integer in [1, 17]
              << " (where we think PI = "  << rng.uniform(3.1,3.2) << ")\n\n" // uniform real in [3.1, 3.2)
              << "We " << rng.pick({"welcome",
                                    "look forward to synergizing with",
                                    "will resist",
                                    "are apathetic towards"})
                   << " our management overlords\n\n";

    std::cout << "On the 'business intelligence' test, we scored "
              << rng.variate(std::normal_distribution<>(70.0, 10.0))
              << "%\n";
}
```

and its current rough equivalent:

```
#include <random>
#include <iostream>
int main()
{
    std::random_device rd; // assume unsigned int is 32 bits
    std::seed_seq sseq {rd(), rd(), rd(), rd(), rd(), rd(), rd(), rd()};
    std::mt19937 engine(sseq); // seeded with 256 bits of entropy from random_device

    auto strings = {"welcome",
                    "look forward to synergizing with",
                    "will resist",
                    "are apathetic towards"
                   };

    std::cout << "Greetings from Office #" << std::uniform_int_distribution<>(1,17)(engine) // uniform integer in [1, 17]
              << " (where we think PI = "  << std::uniform_real_distribution<>(3.1, 3.2)(engine) << ")\n\n" // uniform real in [3.1, 3.2)
           << "We " << *(strings.begin() + std::uniform_int_distribution<>(0, std::size(strings) - 1)(engine))
                      << " our management overlords\n\n";

    std::cout << "On the 'business intelligence' test, we scored "
              << std::normal_distribution<>(70.0, 10.0)(engine)
              << "%\n";
}
```

Putting aside the convoluted seeding procedure for a moment (we note that it would be even more convoluted if manually writing out all the calls is undesirable or impractical, such as when seeding a `std::mt19937` with enough entropy to cover its entire state), it's obvious that even the simplest tasks such as "picking a random element from a list" or "pick a random number from this range" require substantial typing. With the extra complexity, it's also easier to get wrong - accidentally mixing up the two `uniform_{int,real}_distribution`s, or getting the range size wrong for "pick a random element". And there's no easy way to write that `pick`-with-braced-initializer-list line without creating a variable that may never be used again.

Some might object that while `*(strings.begin() + std::uniform_int_distribution<>(0, std::size(strings) - 1)(engine))` seems ugly, `std::uniform_int_distribution<>(1,17)(engine)` is not too unreasonable. But imagine explaining this code to the new programmer: `uniform_int_distribution` is a class template, and the empty angle brackets is needed to tell the compiler to use the default result type (which is `int`); we then construct a temporary `uniform_int_distribution` object passing the desired range as arguments, which can then be called on a random number engine to produce the desired value. That is a lot of scaffolding (template arguments, temporary function objects) to support what is conceptually a simple task ("pick a random integer between 1 and 17"). This complexity makes some users avoid `<random>` completely and write `rand() % 17 + 1` instead - with all the attendant correctness issues. Similar concerns about `<random>`'s lack of approachability motivated `std::experimental::randint` in Library Fundamentals v2 TS, but the facility it provides is limited to uniformly distributed integers and the interface is largely orthogonal to the facilities in `<random>` - it is no easier for a `randint` user to "upgrade" to `<random>` than a `rand` user.

Overview

We propose a class template `random_generator` that operates as a simple wrapper over an underlying URBG, with the following features:

- If the underlying URBG is a random number engine, default construction and calling `.seed()` with no arguments will seed it into an unpredictable state.
- Most operations are supported for all URBGs; the only operations that require a random number engine are the default constructor and `.seed()`
- An in-place constructor is provided that will forward arbitrary arguments to the URBG's constructor, for when the URBG is not an engine or when deterministic seeding is needed.
- A set of member function templates covering the most common tasks used with a random number generator:
  - Making a random selection (`pick`, `choose`)
  - Generating a uniformly distributed value within a range (`uniform`)
  - Shuffling a range (`shuffle`)
  - Sampling a range without replacement (`sample`)

  Together, these covers all of the utility functions offer by Python's `random` package.
- Member functions `generate` and `variate` that allows the user to generate random numbers from an arbitrary user-specified distribution, thus allowing the user to exploit the full power of `<random>`.

Design decisions

Default construction seeds the engine to an unpredictable state

A pain point in using `<random>` is correct nondeterministic seeding. It is therefore crucial to `random_generator`'s design that its default constructor performs this seeding automatically.

This means that the default constructor also requires the underlying URBG type to be a random number engine.

All uniform random number generators are supported

Notably, this includes `random_device`. The only member functions that require engines are the default constructor and `seed()`. For a `random_generator<random_device>`, the constructor that forwards its arguments can be used:

```
random_generator<random_device> rng(std::in_place);
```

The proposed wording reuses `std::in_place_t` as the tag type, but a different one can be used.

Provide convenience member functions for common tasks.

Making a random selection and generating a uniformly distributed value within a range are two of the most common use cases for random number generators and are directly supported by the member function templates `pick`, `choose` and `uniform`. Shuffling and sampling are also supported via the member function templates `shuffle` and `sample`. Together, these covers all of the utility functions offered by Python's `random` package.

Provide a stepping stone to the full C++11 random number generation facility

`random_generator` makes it easy to gradually introduce new users to the full majesty of `<random>`. The simplest uses of `random_generator` will accustom the user to the the idea of generators-as-objects, while the member function templates `variate` and `generate` will introduce the

idea of distributions.

Responses to potential concerns

In your sample code, what's wrong with just writing "`std::mt19937 rng(std::random_device{}());`" (as done in many online examples) and not using `seed_seq`? Aren't you making it more convoluted than necessary?

That seeds the random number generator with a single (probably 32-bit) integer, which means that

- There are only 2^{32} possible initial states for the RNG, so it's very easy to predict future output by searching through them all.
- The first output after the seeding will be biased - more than one-third of the 2^{32} possible outputs will not show up at all regardless of the value of the seed, while others will show up more than once.

There's a proposal already to simplify seeding with `std::random_device`.

We are happy to see that proposal, which goes a long way towards addressing the usability issues with seeding random number engines with `random_device`. However, `std::random_device` is allowed to utilize a random number engine, and is known to do so on at least one platform, and in practice it is difficult for a program to detect whether an engine is being used behind the scenes (`random_device::entropy` always returns zero on several major implementations). Thus, that proposal does not fully address the seeding problem.

But why would an implementation that doesn't support nondeterministic `random_device` support your nondeterministic seeding scheme?

There are other sources of entropy which can be used by the implementation as a fallback for seeding purposes, even though they would not be able to power `std::random_device`. We note that the *per-thread engine* in the Library Fundamentals v2 TS is also required to be initialized to an unpredictable state.

Most functions construct (or encourage the construction of) a distribution object on every call, which can be wasteful. What happened to "you don't pay for what you don't use"?

In our view, for a user of `random_generator`, part of what they are paying for is the convenience, including the convenience of not having to construct a distribution object themselves. Users who desire the very last bit of performance can construct their own distribution objects once and reuse them, possibly with different parameter objects. We do provide a `generate` member function template for the common case of generating multiple random numbers all from the same distribution.

Why member functions? What happened to the lesson of `std::basic_string` and its ~~103~~ ~~127~~ ~~128~~ 130 member functions?

We do agree that `basic_string`'s design is not exactly something that should be emulated. However, we think that `random_generator` here plays a very different role from `basic_string` that make the analogy inapposite. (Several of the points below were raised by Zhihao Yuan on std-proposals, with which we agree.)

First, `random_generator` is a thin glue layer that wraps any URBG, in the standard library or outside. `std::basic_string` is a concrete string class, with many analogues in other code bases.

Second, unlike things like `find_first_of` or `replace`, which applies equally well to all sequences, including non-`char` ones, `pick`, `choose`, `uniform`, etc. don't really apply outside the random number context. Our `generate` is not `std::generate`.

Third, the design of `random_generator` intentionally favors convenience over maximal efficiency (see the previous question), so our member functions are not necessarily the most efficient - for instance, calling `pick`, `choose`, and `uniform` multiple times may not be as efficient as the hand-written version if the distribution object is stateful. Packaging them together provides a simple rule of thumb - "if you care about getting every last bit of performance, don't use `random_generator`", which seems preferable to "if you care about getting every last bit of performance, don't use these X different things".

We also note that with member functions, the name of the class provides context, allowing shorter names such as `pick` and `choose` to be used without causing ambiguity. Unlike `shuffle` and to a lesser degree `sample`, words like `pick`, `choose` or `uniform` do not inherently bring randomness to mind: it is far from obvious that a hypothetical `std::pick` means "pick randomly", whereas `random_generator::pick` clearly implies that the picking is done in a random manner.

Finally, we simply do not have that many member functions - our member function count is 20, not 109 :)

Do we really need both `pick` and `choose`?

`pick` deals with containers, and it's natural for it to return a reference to the element picked; `choose` is passed a pair of iterators, so it's natural for it to return an iterator. We think both are useful and worth providing. `pick()` is sugar for `*choose()` much like `front()` is sugar for `*begin()`.

Impact on the standard

This is a pure extension.

Technical specification

Open questions

- Is there a better way to specify the range-based overloads other than *BEGIN_EXPR* / *END_EXPR*?

Addition to `<random>` synopsis

```
template <class URBG = default_random_engine>
class random_generator;
using default_rng = random_generator<>;
using mt19937_rng = random_generator<mt19937>;
using mt19937_64_rng = random_generator<mt19937_64>;
```

Class template `random_generator` synopsis

> *Drafting note*: the name `URBG` brings in the requirement in [rand.req.genl].

```
template <class URBG = default_random_engine>
class random_generator {
public:
    using urbg_type       = URBG;
private:
    urbg_type urbg_; // exposition only
public:
    random_generator();

    template <class... Params>
    explicit random_generator(in_place_t, Params&&... params);

    void seed();

    template <class... Params>
    void seed(Params&&... params);

    URBG& urbg() noexcept;

    template <class DistType>
    typename DistType::result_type variate(DistType&& dist);

    template <class Numeric>
    Numeric uniform(Numeric lower, Numeric upper);

    template <class DistType, class ForwardIterator>
    void generate(DistType&& dist, ForwardIterator first, ForwardIterator last);

    template <class DistType, class Range>
    void generate(DistType&& dist, Range&& range);

    template <class ForwardIterator>
    void generate(ForwardIterator first, ForwardIterator last);

    template <class Range>
    void generate(Range&& range);

    template <class RandomAccessIterator>
    void shuffle(RandomAccessIterator first, RandomAccessIterator last);

    template <class Range>
    void shuffle(Range&& range);

    template <class ForwardIterator>
    ForwardIterator choose(ForwardIterator first, ForwardIterator last);

    template <class Range>
    auto choose(Range&& range);

    template <class Range>
    decltype(auto) pick(Range&& range);

    template <class T>
    decltype(auto) pick(std::initializer_list<T> range);

    template <class ForwardIterator, class Distance>
    ForwardIterator inplace_sample(ForwardIterator first, ForwardIterator last, Distance n);

    template <class Range, class Distance>
    auto inplace_sample(Range&& range, Distance n);
```

```
    template <class PopulationIterator, class SampleIterator, class Distance>
    SampleIterator sample(PopulationIterator first, PopulationIterator last, SampleIterator out, Distance n);

};
```

`random_generator` construction and seeding

```
random_generator();
```

Requires: `URBG` shall satisfy the random number engine requirements ([rand.req.eng]).

Effects: Initializes `urbg_` to an unpredictable state.

Remarks: Implementations should seed `urbg_` using at least 256 bits of entropy of reasonable quality. [*Note*: the seeding need not be cryptographically secure, but should not be trivially weak. — *end note*] Implementations should ensure that distinct calls to this constructor and to `seed()` utilize distinct seeds, even if the calls are closely separated in time.

```
template <class... Params>
explicit random_generator(in_place_t, Params&&... params);
```

Effects: Direct-non-list-initializes `urbg_` with `std::forward<Params>(params)...`.

Remarks: No additional seeding is performed. [*Note:* If `URBG` satisfies the random number engine requirements ([rand.req.eng]), the underlying engine may be seeded by calling `seed()`. — *end note*]

```
void seed();
```

Requires: `URBG` shall satisfy the random number engine requirements ([rand.req.eng]).

Effects: Reseeds `urbg_` to an unpredictable state.

Remarks: Implementations should seed `urbg_` using at least 256 bits of entropy of reasonable quality. [*Note*: the seeding need not be cryptographically secure, but should not be trivially weak. — *end note*] Implementations should ensure that distinct calls to `seed()` and to the default constructor utilize distinct seeds, even if the calls are closely separated in time.

```
template <class... Params>
void seed(Params&&... params);
```

Requires: `URBG` shall satisfy the random number engine requirements ([rand.req.eng]).

Effects: Equivalent to `urbg_.seed(std::forward<Params>(params)...);`

`random_generator` underlying engine access

```
URBG& urbg() noexcept;
```

Returns: `urbg_`.

`random_generator` random number generation

Given an expression `t`, define *`BEGIN_EXPR`*`(t)` and *`END_EXPR`*`(t)` as follows:

- If the expression `std::begin(t)` is valid ([temp.deduct]) when considered as an unevaluated operand, then *`BEGIN_EXPR`*`(t)` is `std::begin(t)` and *`END_EXPR`*`(t)` is `std::end(t)`. [*Note:* This handles classes with member `begin()` and `end()`. — *end note*]
- Otherwise, *`BEGIN_EXPR`*`(t)` is `begin(t)` and *`END_EXPR`*`(t)` is `end(t)`, where `begin` and `end` are looked up in the associated namespaces ([basic.lookup.argdep]) of `T`. [*Note:* This can be done by performing an unqualified call in a context where no viable `begin` or `end` are found by ordinary unqualified lookup. — *end note*]

> *Drafting note*: the intended semantics here is "use `std::begin` if valid, ADL `begin` otherwise". (The `std::begin` that dispatches to `.begin()` is SFINAE-friendly.) Done in two steps so as to avoid ambiguities due to possible greedy ADL `begin`s.

```
template <class DistType>
typename remove_reference_t<DistType>::result_type variate(DistType&& dist);
```

Requires: `remove_reference_t<DistType>` shall satisfy the random number distribution requirements ([rand.req.dist]).

Effects: Equivalent to `return dist(urbg_);`

```
template <class Numeric>
Numeric uniform(Numeric lower, Numeric upper);
```

Let `UniformDist` be `uniform_int_distribution<Numeric>` if `Numeric` is an integral type, `uniform_real_distribution<Numeric>` otherwise. [*Note:* This implies that `Numeric` must be one of the allowed types for `IntType` or `RealType` in [rand.req.genl]. — *end note*]

Effects: Equivalent to `return variate(UniformDist(lower, upper));`

```
template <class DistType, class ForwardIterator>
void generate(DistType&& dist, ForwardIterator first, ForwardIterator last);

template <class ForwardIterator>
void generate(ForwardIterator first, ForwardIterator last);
```

For the second overload, let `T` be the value type of `ForwardIterator`, and let `dist` be `uniform_int_distribution<T>()` if `T` is an integral type, and `uniform_real_distribution<T>()` otherwise. [*Note:* This implies that `T` must be one of the allowed types for `IntType` or `RealType` in [rand.req.genl]. — *end note*]

Requires: `ForwardIterator` shall meet the requirements of a forward iterator ([forward.iterators]). For the first overload, `remove_reference_t<DistType>` shall satisfy the random number distribution requirements ([rand.req.dist]).

Effects: Equivalent to:

```
auto&& d_ = dist;
std::generate(first, last, [&]{return d_(urbg_);});
```

```
template <class DistType, class Range>
void generate(DistType&& dist, Range&& range);
```

Effects: Equivalent to `generate(dist, `*`BEGIN_EXPR`*`(range), `*`END_EXPR`*`(range));`

```
template <class Range>
void generate(Range&& range);
```

Effects: Equivalent to `generate(`*`BEGIN_EXPR`*`(range), `*`END_EXPR`*`(range));`

`random_generator` shuffling and sampling

```
template <class RandomAccessIterator>
void shuffle(RandomAccessIterator first, RandomAccessIterator last);
```

Requires: `RandomAccessIterator` shall meet the requirements of a random access iterator ([random.access.iterators]).

Effects: Equivalent to `std::shuffle(first, last, urbg_);`

```
template <class Range>
void shuffle(Range&& range);
```

Effects: Equivalent to `shuffle(`*`BEGIN_EXPR`*`(range), `*`END_EXPR`*`(range));`

```
template <class ForwardIterator>
ForwardIterator choose(ForwardIterator first, ForwardIterator last);
```

Requires: `ForwardIterator` shall meet the requirements of a forward iterator ([forward.iterators]).

Returns: If `first == last`, `first`. Otherwise, an iterator in the range `[first, last)`, such that each iterator in the range has equal probability of being chosen.

Remarks: To the extent that the implementation of this function makes use of random numbers, the object `urbg_` shall serve as the implementation's source of randomness.

```
template <class Range>
auto choose(Range&& range);
```

Effects: Equivalent to `return choose(`*`BEGIN_EXPR`*`(range), `*`END_EXPR`*`(range));`

```
template <class Range>
decltype(auto) pick(Range&& range);

template <class T>
decltype(auto) pick(std::initializer_list<T> range);
```

Effects: Equivalent to `return *choose(range);;`

```
template <class ForwardIterator, class Distance>
ForwardIterator inplace_sample(ForwardIterator first, ForwardIterator last, Distance n);
```

Requires: `ForwardIterator` shall meet the requirements of a forward iterator ([forward.iterators]) and the `ValueSwappable` requirements ([swappable.requirements]). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table [tab:moveconstructible]) and `MoveAssignable` (Table [tab:moveassignable]).

In the description below, the + and - operators have the semantics specified in [algorithms.general].

Effects: If `n >= last - first`, has no effects. Otherwise, reorders elements in the range `[first, last)`, such that each possible sample of size `n` has equal probability of appearing in the range `[first, first + n)`. The relative order of the elements both inside the sample and outside the sample is preserved.

Returns: `first + min(n, last - first)`.

Remarks: To the extent that the implementation of this function makes use of random numbers, the object `urbg_` shall serve as the implementation's source of randomness.

```
template <class Range, class Distance>
auto inplace_sample(Range&& range, Distance n);
```

Effects: Equivalent to `return inplace_sample(`*`BEGIN_EXPR`*`(range), `*`END_EXPR`*`(range), n);`

```
template <class PopulationIterator, class SampleIterator, class Distance>
SampleIterator sample(PopulationIterator first, PopulationIterator last, SampleIterator out, Distance n);
```

Effects: Equivalent to `return std::sample(first, last, out, n, urbg_);`

Acknowledgements

We thank Chandler Carruth for his encouragement and support. We also thank Zhihao Yuan, Jeffrey Yasskin, Nicol Bolas, and other members of the std-proposal mailing list for their comments on an earlier draft of this proposal.